Quickdress: Design Specification

Milestone 1 · Scoping, Requirements and Architecture · Revised November 5, 2026
Prepared by Marwan Abouelfotuh
Team: Aya Noaman, Malk Metwally, Nada Salem, Marwan Abouelfotuh, Omar Hassan

This document is the companion to the Quickdress Requirements Specification. Requirement IDs (FR-xx, NFR-xx) refer to that document.

What changed in this revision

The first version (October 13, 2026) received two main comments from the instructor. Table 0 shows how this revision answers each one.

Comment	What we changed
A layered architecture is too simple for our app.	The app is now a **microkernel (plug-in) architecture**: a small core plus plug-ins for capture, storage, context, filters and recommenders, with an in-process event bus. Outfit generation is a pipe-and-filter pipeline inside the core. MVVM stays as the design of the screens. Every diagram, table and section that described the layers was rewritten (sections 1 to 7).
We make statements that we do not support with concrete plans.	Every promise now has a rule, a number or a test behind it: the scoring rules and thresholds (section 2.2), the formality keywords (section 2.2), a test plan with a sprint, a tool and a pass mark for each non-functional requirement (Table 6), exit criteria and a decision rule for stretch goals in the build order (Table 8), and a decision point for every risk (Table 9). The Core ML recommender, which had no data or plan behind it, was removed from this term's design.

Table 0. Instructor comments and the changes made.

Contents

1. High-level architecture
2. Explanation of the architecture
3. Quality attributes and how we will test them
4. Architectural and design patterns
5. Detailed design: class diagram
6. Technical feasibility, build order and risks
7. Conclusion

1 High-level architecture

1.1 Overview

Quickdress is a single iOS app with no backend of its own. Wardrobe data, photos and calendar information stay on the phone. The app makes only two kinds of network call: one to a weather service for the forecast and, if the user opts in (FR-22), one to Google's Gemini 3.1 Flash-Lite model through Firebase AI Logic.

The app uses a **microkernel architecture**, also called a plug-in architecture. It has three parts:

- **The core system.** A small set of components that hold the rules that never change: what a valid clothing item is, how the day's context is put together, and how an outfit is produced. The core also owns the **plug-in registry**, where plug-ins are registered at launch, and the **event bus**, which tells interested components when something has happened. The core is plain Swift and imports no Apple framework apart from Foundation.
- **Plug-ins.** Everything that can vary or be replaced: where photos come from (camera or photo library), where data is stored (SwiftData, files), where the day's information comes from (WeatherKit, Open-Meteo, the calendar), which filters an outfit must pass, and which recommender picks the outfit (rule-based or Gemini). Each plug-in implements a contract (a Swift protocol) defined by the core and never calls another plug-in.
- **The host app.** The SwiftUI screens, built with MVVM (Model, View, ViewModel), and the composition root (`QuickdressApp`), which creates every plug-in and registers it with the core. ViewModels call only the core's public services.

The reason for this structure is that most of what is hard about Quickdress is choosing between interchangeable parts at run time. There are two weather sources (R1), two recommenders with a fallback between them (FR-22, NFR-03), two image sources, four outfit filters, and more parts that the backlog already plans for later, such as a laundry filter (FR-20) and iCloud storage (FR-21). In a microkernel these are all the same kind of thing: a plug-in with a priority and an availability check. The core decides which one to use, so adding or removing one does not change the core or any screen (NFR-08).

iOS does not allow an app to download and run new code (App Store Review Guideline 2.5.2), so our plug-ins are not loaded at run time. Each plug-in group is its own Swift package, compiled into the app and registered in code at launch. What makes it a plug-in is that the core depends only on the contract, and the plug-in is chosen by the registry, not by the caller.

1.2 Component diagram

Quickdress: UML component diagram

Microkernel architecture: a Foundation-only core with a plug-in registry and event bus; plug-ins realise the core's contracts. Arrows point from the component that depends on another to the component it uses.

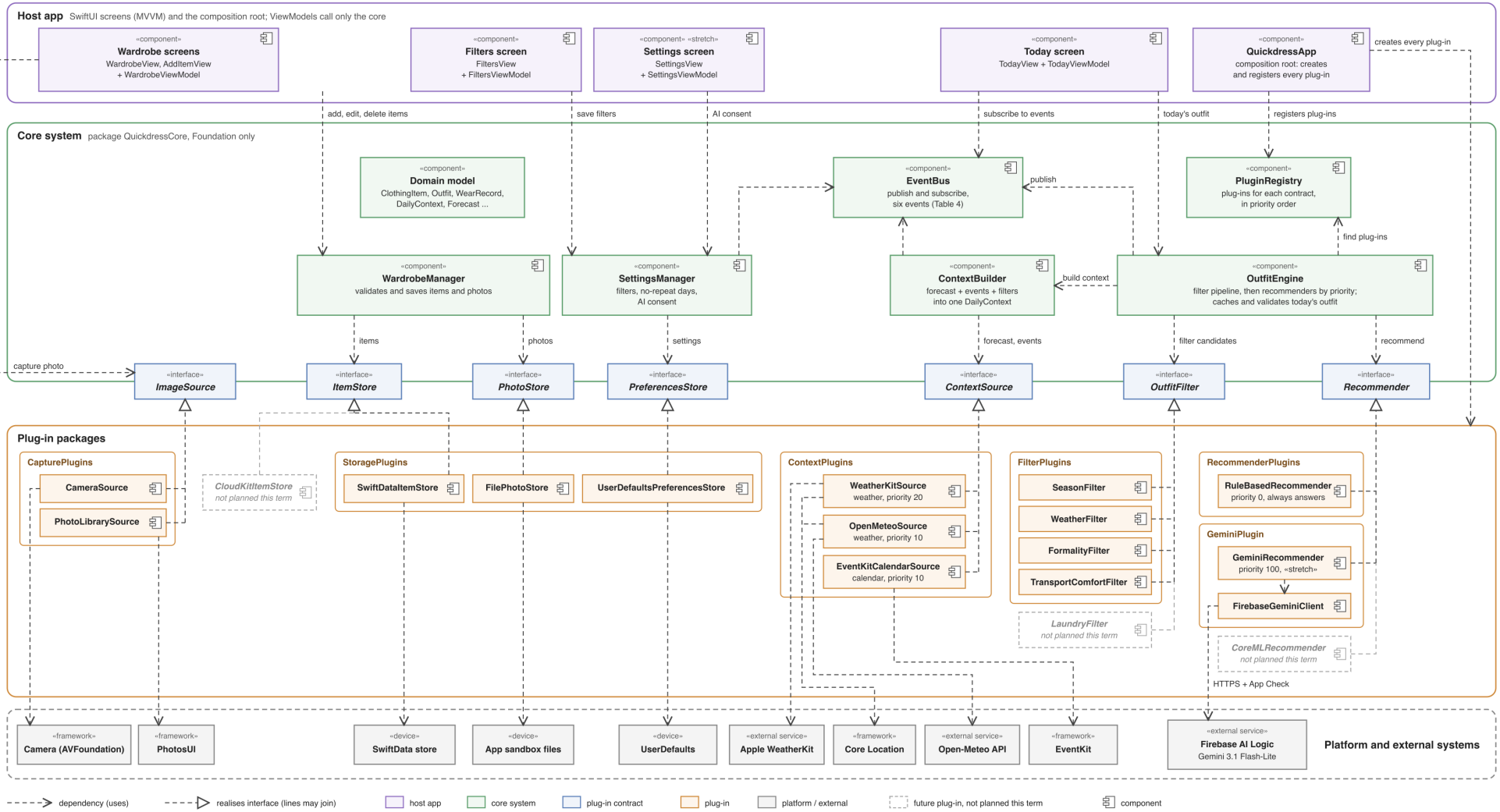


Figure 1. UML component diagram of Quickdress: the core system, its plug-in contracts, the plug-ins that realise them, and the host app.

1.3 The core system

Table 1 lists the components of the core. All of them live in one Swift package, `QuickdressCore`, which has no dependency on any other package in the project. The compiler therefore stops the core from using a plug-in directly.

Component	Responsibility	Uses	Requirements
PluginRegistry	Holds every registered plug-in. Returns the plug-ins for a contract in priority order, filtered to those that say they are available. For contracts that need exactly one implementation (the stores), it returns that one and fails a launch test if it is missing.	Plug-in contracts	NFR-08
EventBus	Delivers events (Table 4) to subscribers, on the main thread, in the order they subscribed. Events only report that something happened. They never carry a request that needs an answer.	None	FR-09, NFR-01
WardrobeManager	Adds, edits and deletes items. Checks that the category is set, the colour is one of the 12 palette colours, ratings are 1 to 5 and at least one season is chosen. Saves the photo first and the item second, and deletes the photo again if the item fails to save. Publishes `itemSaved` and `itemDeleted`.	ItemStore, PhotoStore, EventBus	FR-01 to FR-03, NFR-06
ContextBuilder	Builds one `DailyContext` for a date: forecast, events, filters and the day's formality level. For each kind of context it tries the context plug-ins in priority order, with a 3-second limit each, and falls back to cached or default values (section 2.1). Publishes `contextDegraded` when it had to fall back.	Context plug-ins, PreferencesStore, EventBus	FR-08, FR-10, FR-11, NFR-03, NFR-07
OutfitEngine	Produces, caches and changes the day's outfit. Runs the filter pipeline, asks the recommender plug-ins in priority order, checks every answer, handles swaps and marks outfits as worn (section 2.2). Publishes `outfitGenerated` and `outfitWorn`.	ContextBuilder, filter and recommender plug-ins, ItemStore, EventBus	FR-05 to FR-07, FR-09, NFR-01, NFR-03, NFR-05
SettingsManager	Reads and saves the daily filters, the no-repeat period (default 7 days, 0 to 30) and the AI consent switch. Publishes `consentChanged`.	PreferencesStore, EventBus	FR-09, FR-10, FR-22, NFR-02
Domain model	Plain Swift value types shared by the core and the plug-ins: `ClothingItem`, `Outfit`, `WearRecord`, `DailyContext`, `DailyFilters`, `Forecast`, `CalendarEvent`, and the enumerations `Category`, `Palette` and `Season`. They are not SwiftData classes, so the core does not depend on any storage framework.	None	FR-02

Table 1. Components of the core system.

1.4 Plug-in contracts and the plug-ins that implement them

The core defines five kinds of plug-in contract, made of seven protocols because storage has three. Every plug-in has an `id` and a `priority`, and most have an availability check. Table 2 lists each contract, its operations and how the core chooses between implementations.

Contract	Main operations	How the core uses it
ImageSource (capture)	`isAvailable`, `requestAccess()` `async -> Bool`, `capture()` `async throws -> Data` (JPEG)	The Add item screen lists every available source as a button. The source the user taps is used.
ItemStore, PhotoStore, PreferencesStore (storage)	`fetchItems()`, `save(_:)`, `delete(_:)`, `fetchWearLog(since:)`, `logWorn(_:on:)`; `savePhoto(_:)` `-> PhotoRef`, `loadPhoto(_:)`, `deletePhoto(_:)`; `filters`, `noRepeatDays`, `aiConsent`	Exactly one of each is registered.
ContextSource (context)	`facet` (weather or calendar), `isAvailable()`, `contribute(for: Date) async throws`	For each facet, sources are tried from the highest priority down. The first one that answers within 3 seconds is used.
OutfitFilter (filters)	`apply(_ candidates:, context:) -> Candidates`	Every filter runs, lowest priority number first, as one stage of the pipeline. A filter may not empty a slot (section 2.2).
Recommender (recommenders)	`isAvailable(_ environment:)` `async -> Bool`, `recommend(_ request:) async throws -> Outfit`	Tried from the highest priority down. The first valid answer is used. The rule-based recommender has the lowest priority and is always available.

Table 2. Plug-in contracts.

Table 3 lists the plug-ins. Each row says which contract it implements, what it wraps, its priority, and when it is built. Plug-ins marked as stretch are built only under the decision rule in section 6.2. The last three rows are plug-ins the design allows for but that we do not plan to build this term.

Plug-in	Contract	Wraps	Priority	Built in	Requirements
CameraSource	ImageSource	Camera (AVFoundation, through `UIImagePickerController`)	n/a	Sprint 1	FR-01, NFR-07
PhotoLibrarySource	ImageSource	PhotosUI `PhotosPicker` (needs no Photos permission)	n/a	Sprint 1	FR-01
SwiftDataItemStore	ItemStore	SwiftData, with its own `@Model` records mapped to the domain types	n/a	Sprint 1	FR-02, FR-03, FR-09, NFR-06
FilePhotoStore	PhotoStore	Files in Application Support	n/a	Sprint 1	FR-01, NFR-06
UserDefaultsPreferencesStore	PreferencesStore	UserDefaults	n/a	Sprint 2	FR-09, FR-10, FR-22

Plug-in	Contract	Wraps	Priority	Built in	Requirements
WeatherKitSource	ContextSource (weather)	WeatherKit, plus reduced-accuracy Core Location	20	Sprint 2, if R1 allows	FR-08
OpenMeteoSource	ContextSource (weather)	Open-Meteo HTTP API, plus reduced-accuracy Core Location	10	Sprint 2	FR-08, NFR-03
EventKitCalendarSource	ContextSource (calendar)	EventKit, read-only	10	Sprint 3	FR-11, NFR-07
SeasonFilter	OutfitFilter	None	10	Sprint 2	FR-05
WeatherFilter	OutfitFilter	None	20	Sprint 2	FR-08
FormalityFilter	OutfitFilter	None	30	Sprint 3	FR-10, FR-11
TransportComfortFilter	OutfitFilter	None	40	Sprint 3	FR-10
RuleBasedRecommender	Recommender	None (scoring in plain Swift)	0 (last)	Sprint 2	FR-05 to FR-07, NFR-03, NFR-05
GeminiRecommender	Recommender	Firebase AI Logic (Gemini 3.1 Flash-Lite)	100 (first)	Sprint 3, stretch	FR-22, NFR-02
ItemAnalyzer	None (a helper in CapturePlugins used by the Add item screen)	Core Image and Vision, on the device	n/a	Sprint 3, stretch	FR-13
CoreMLRecommender	Recommender	An on-device model	50	Not planned (no training data)	FR-16
LaundryFilter	OutfitFilter	None	50	Not planned (FR-20 deferred)	FR-20
CloudKitItemStore	ItemStore	CloudKit	n/a	Not planned (FR-21 deferred)	FR-21

Table 3. Plug-ins. Higher priority is tried first for context sources and recommenders; for filters, lower numbers run first.

1.5 Rules that keep the structure intact

1. **The core depends on nothing but Foundation.** `QuickdressCore` is a Swift package with no package dependencies. A plug-in package depends on `QuickdressCore` and on the one Apple or Google SDK it wraps. The host app depends on all of them. Because Swift packages cannot have circular dependencies, the build fails if the core ever tries to use a plug-in.
2. **Plug-ins never call each other.** Anything a plug-in needs from another part of the app comes in through the request the core passes to it. The two weather plug-ins share one

helper, `CoreLocationProvider`, which lives inside the context plug-in package and is not a plug-in itself.

3. **One composition root.** `QuickdressApp.init()` creates the plug-ins, registers them and creates the core services. Tests and SwiftUI previews register fakes in the same way. A launch test fails if a required contract (the three stores) has no plug-in.
4. **Commands are calls, notifications are events.** When a ViewModel needs an answer (today's outfit, the item list) it calls a core service and waits for the result. When something has happened that other parts may care about (an item was deleted), the core publishes an event. An event handler may not publish an event of the same type, which rules out loops.
5. **ViewModels see only the core.** They call `WardrobeManager`, `OutfitEngine` and `SettingsManager`, subscribe to the event bus, and ask the registry for the list of image sources. They never import a plug-in package.

1.6 Events

Table 4 lists every event, who publishes it and what each subscriber does with it.

Event	Published by	Subscribers and what they do
`itemSaved(id)`	WardrobeManager	OutfitEngine: if there is no outfit today because a slot was empty, it generates one now. WardrobeViewModel: reloads the grid.
`itemDeleted(id)`	WardrobeManager	OutfitEngine: if the item is in today's outfit, it generates a new outfit. WardrobeViewModel: reloads the grid.
`outfitGenerated(outfit, by:)`	OutfitEngine	TodayViewModel: shows the outfit. The by field names the recommender, used only for logging.
`outfitWorn(outfit, date)`	OutfitEngine	TodayViewModel: shows the outfit as worn. WardrobeViewModel: updates the last-worn date on each item.
`consentChanged(on)`	SettingsManager	GeminiRecommender: reports itself available or unavailable from now on.
`contextDegraded(reason)`	ContextBuilder	TodayViewModel: shows the offline badge, or the note that the outfit ignores the weather or the calendar.

Table 4. Events on the event bus.

2 Explanation of the architecture

This section follows the two most important requests through the system: viewing today's outfit, and producing an outfit inside the engine. It then covers adding an item, the Gemini plug-in, and the alternatives we rejected.

2.1 Viewing today's outfit

Quickdress: sequence diagram for "View today's outfit"

Solid arrows are calls, dashed arrows are returns. Message numbers match the steps in section 2.1. Purple: screens, green: core, orange: plug-ins.

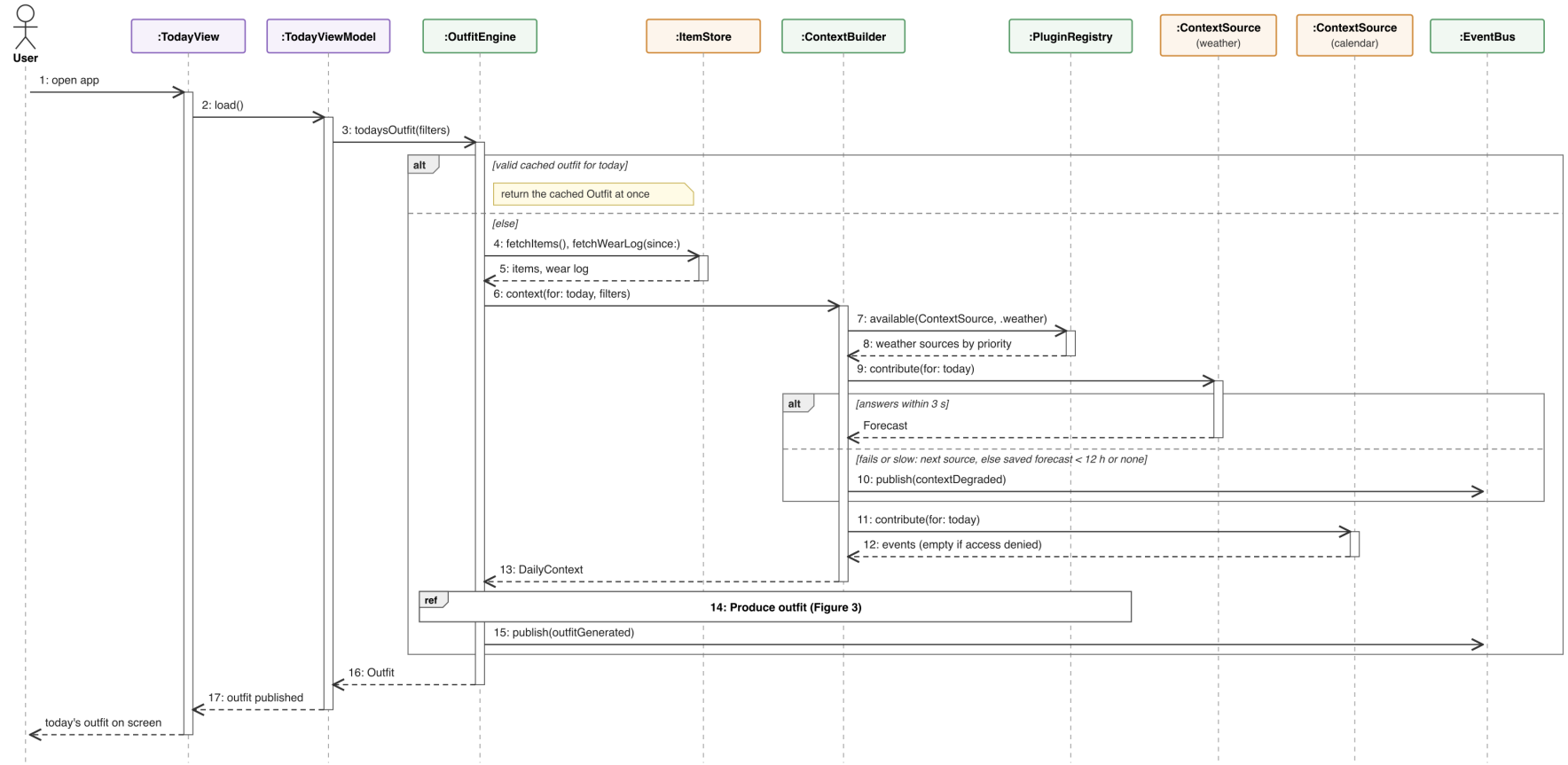


Figure 2. Sequence diagram for "View today's outfit" (FR-05, FR-08, FR-11, NFR-01, NFR-03). The step inside the engine is shown in Figure 3.

Figure 2 shows the main use case. The numbers in brackets are the message numbers in the figure.

1. The user opens the app and `TodayViewModel.load()` calls `OutfitEngine.todaysOutfit(filters)` (1 to 3).
2. If an outfit was already generated today and nothing it depends on has changed, the engine returns it from its cache at once. The cache is saved through the ItemStore, so it survives a relaunch and the screen is never blank (NFR-01). The rest of the steps run only when there is no valid cached outfit.
3. The engine loads the items and the wear log for the no-repeat period from the ItemStore (4, 5).
4. It asks `ContextBuilder` for today's `DailyContext` (6). The builder asks the registry for the available weather sources in priority order (7, 8) and calls the first one (9). If it fails or takes more than 3 seconds, the builder tries the next one. If every source fails, it uses the last saved forecast if it is less than 12 hours old, and otherwise uses no forecast. Either way it publishes `contextDegraded`, and the Today screen shows a badge (10). If location permission is denied, the weather sources report themselves unavailable and the outfit is chosen for mild weather, with a note on screen.
5. The builder reads today's events from the calendar source (11, 12). If calendar access is denied, the list is empty and formality comes only from the filters (NFR-07). It then works out the day's formality level (section 2.2) and returns the context (13).
6. The engine runs the pipeline and the recommenders (14), shown in detail in Figure 3, saves the result as today's outfit and publishes `outfitGenerated` (15). The ViewModel returns the outfit to the view, which draws it (16, 17).

Swapping a piece (FR-06) calls `OutfitEngine.alternatives(for: slot)`. The engine returns the next three items for that slot in the rule-based ranking, keeping the other pieces fixed. Swaps never call Gemini, so they are instant and free. Asking for a new outfit (FR-07) runs the pipeline again with the current outfit added to the excluded combinations. Marking an outfit as worn (FR-09) calls `OutfitEngine.markWorn()`, which writes a `WearRecord` through the ItemStore and publishes `outfitWorn`.

2.2 Inside the outfit engine: the pipeline and the recommender plug-ins

The engine is a pipe-and-filter pipeline. Each stage takes the candidate items for every slot and passes a smaller set to the next stage. Figure 3 shows both cases: Gemini available and answering, and Gemini off, offline or failing, when the rule-based plug-in answers.

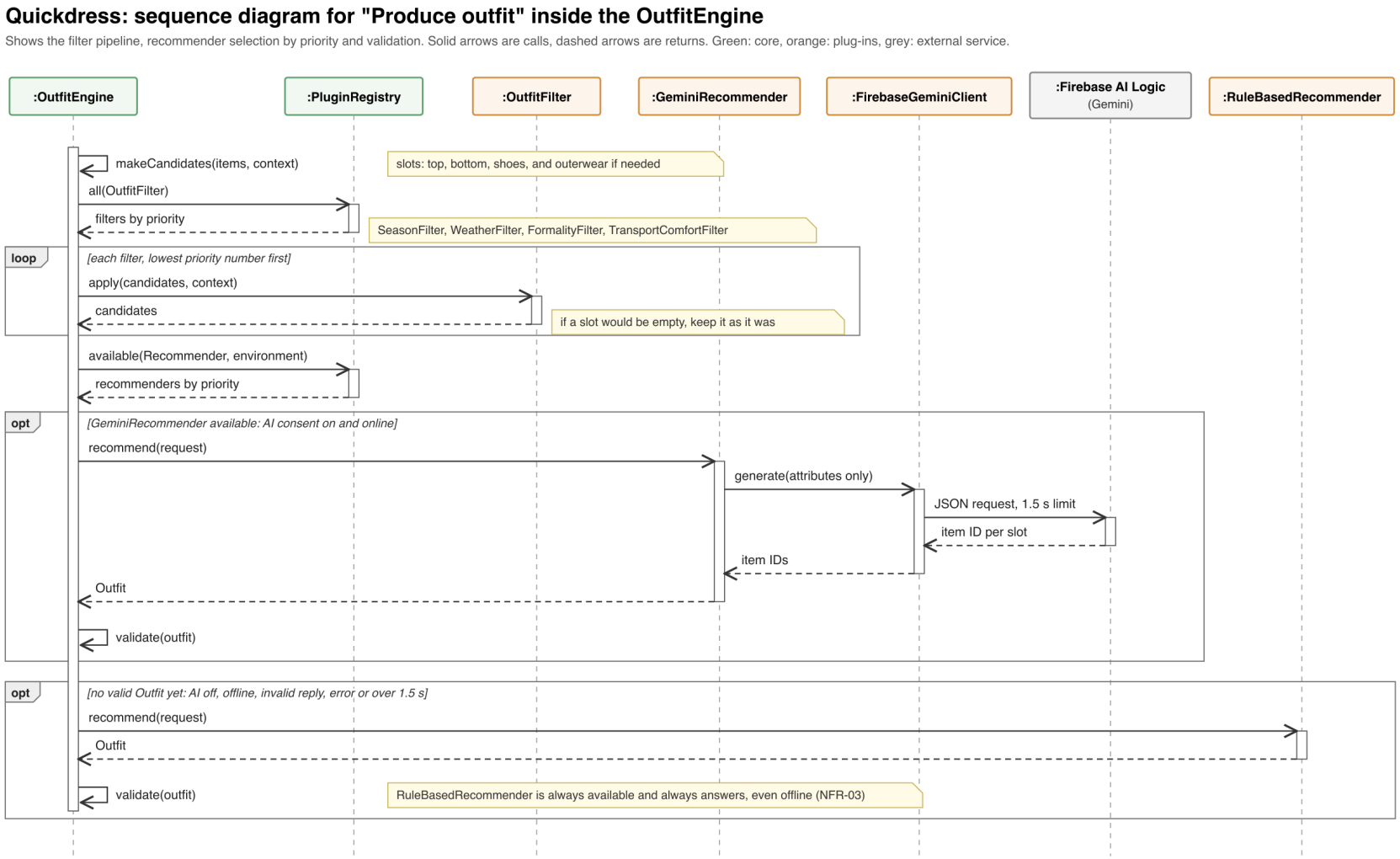


Figure 3. Sequence diagram for "Produce outfit" inside the OutfitEngine: filter pipeline, recommender selection by priority, and validation (FR-05, FR-08 to FR-11, FR-22, NFR-02, NFR-03, NFR-05, NFR-08).

Stage 1: candidates

Items are grouped into slots: top, bottom, shoes and, when needed, outerwear. Accessories are not part of this term's outfits. Outerwear is needed when the day's average apparent temperature is below 17 °C or the chance of rain is 50% or more.

Stage 2: filters

Every registered filter runs in priority order. If a filter would leave a slot empty, the engine keeps that slot as it was before the filter and notes that the filter was relaxed. This is why the app always produces an outfit as long as the wardrobe has at least one top, one bottom and one pair of shoes.

Filter	Rule
SeasonFilter	Keeps items whose seasons include the current one. Seasons follow the calendar months: December to February is winter, March to May spring, June to August summer and September to November autumn.
WeatherFilter	Works out a target warmth level from the average apparent (feels-like) temperature between 08:00 and 20:00: 30 °C and above gives 1, 24 to 29 °C gives 2, 17 to 23 °C gives 3, 10 to 16 °C gives 4, and below 10 °C gives 5. Keeps tops, bottoms and outerwear within one level of the target. With no forecast, the target is 3.
FormalityFilter	Keeps items within one level of the day's formality. The day's formality is the override from the filters if the user set one. Otherwise it is 4 if any event today is tagged formal by the user or its title contains a formality keyword, and 2 if not.
TransportComfortFilter	When the transport mode is walking or public transport, removes shoes with comfort below 3. When the comfort priority is high, removes every item with comfort below 3.

Table 5. The four filters in pipeline order.

The formality keywords are kept in one file, `FormalityKeywords.swift`, and matching ignores case. The first list is: interview, presentation, defense, defence, viva, conference, ceremony, graduation, wedding, formal, and the Arabic words مقابلة (interview), عرض (presentation), مؤتمر (conference), حفل (ceremony), تخرج (graduation) and فرح (wedding). Risk R7 covers the cases this list will miss.

Stage 3: recommenders

The engine builds an `OutfitRequest` with the filtered candidates, the context, and the combinations worn within the no-repeat period, which are excluded. It then asks the available recommenders in priority order. Each answer is checked: every slot that is needed must have exactly one item, every item must come from the candidates, and the combination must not be excluded. If an answer fails the check, throws an error or takes longer than 1.5 seconds, the engine asks the next recommender. The rule-based recommender is always last and always answers, so the app needs no network to produce an outfit (NFR-03).

The rule-based recommender scores each candidate on its own, keeps the best five per slot, and then scores the combinations:

- **Item score** = 0.4 × warmth fit + 0.3 × formality fit + 0.2 × (comfort ÷ 5) + 0.1 × freshness. Warmth fit is 1 minus the gap between the item's warmth and the target, divided by 4. Formality fit works the same way. Freshness is the number of days since the item was last worn, capped at 14 and divided by 14; an item never worn scores 1.
- **Combination score** = 0.7 × the average item score + 0.3 × the colour score. The colour score is the average over every pair of pieces in the outfit, read from a 12 × 12 colour table.

Any colour paired with a neutral (black, white, grey, navy, beige) scores 1.0. Two pieces of the same non-neutral colour score 0.6. Pairs listed as clashing (red with pink, red with green, orange with pink, purple with yellow) score 0.2. Every other pair scores 0.5. The full table is written in Sprint 2 in `ColourTable.swift`.

- Five candidates in each of four slots give at most 625 combinations, however large the wardrobe is, which keeps generation under the one-second limit in NFR-05. Ties go to the combination with the higher freshness.

The weights above are the starting values. They are constants in one file, `ScoringWeights.swift`, and the team changes them only once, after the Sprint 2 usability test (Table 6), based on which outfits the testers rejected.

Worked example. The forecast gives an average apparent temperature of 21 °C with a 10% chance of rain, and the calendar has "Group presentation" at 10:00. The target warmth is 3, no outerwear is needed, and the formality is 4 because of the keyword "presentation". SeasonFilter, WeatherFilter and FormalityFilter leave, say, 6 tops, 4 bottoms and 3 pairs of shoes. If Gemini is turned off or the phone is offline, the rule-based recommender scores the 6 × 4 × 3 = 72 combinations and returns the best one that was not worn in the last 7 days.

2.3 Adding a clothing item

The Add item screen asks the registry for the available image sources and shows one button for each. On the Simulator, `CameraSource` reports itself unavailable, so only the photo library is offered (R10). When the user taps a source, the source asks for permission if it needs it (the camera does, `PhotosPicker` does not) and returns the photo as JPEG data, scaled so that its long edge is at most 1600 pixels. If the stretch analyzer is built (FR-13), it suggests a category and a colour, which the user confirms or changes. The user sets the other attributes and taps Save.

`WardrobeManager.addItem` validates the attributes, asks the PhotoStore to write the photo and gets back a reference, then saves the item with that reference through the ItemStore. If saving the item fails, the manager deletes the photo it has just written, so no orphan files are left behind (NFR-06). It then publishes `itemSaved`. Deleting an item runs in the opposite order and publishes `itemDeleted`, which makes the engine replace today's outfit if the item was in it. The detailed design of the capture screens is a separate deliverable.

The Gemini plug-in and what it sends

`GeminiRecommender` reports itself available only when the AI switch is on and the phone is online. The switch is off by default, and the Settings screen explains in one sentence what is sent before the user can turn it on. This is the explicit consent that NFR-02 requires. When it runs, its client, `FirebaseGeminiClient`, sends:

- for each candidate item that passed the filters: its ID, category, colour, warmth, formality, comfort and seasons;
- for the day: the target warmth, whether outerwear is needed, the formality level, the transport mode and the comfort priority;
- the excluded combinations, as lists of item IDs.

It never sends photos, item names, calendar event titles or the user's location. Gemini is asked for structured JSON output that names one item ID per slot. The engine's check (stage 3) rejects any reply that does not fit. The model name is stored in Firebase Remote Config, so it can be

changed without an app update. The plug-in uses Firebase AI Logic rather than calling the Gemini API directly, because a direct call would mean shipping an API key inside the app, where it can be extracted. Firebase keeps the key on Google's side and uses App Check (App Attest on iOS) to accept requests only from genuine copies of our app.

2.4 Alternatives considered

Alternative	Why we did not choose it
Layered architecture (our first version)	It grouped components by technical level, but it had no place for the main thing Quickdress does: choosing between interchangeable sources and recommenders at run time, with a fallback when one fails. That logic was spread across a relaxed set of layers in which ViewModels could skip the domain layer. The instructor also judged it too simple for the app.
Hexagonal (ports and adapters)	Close to what we have, but it expects one adapter per port, chosen when the app is built. We need several plug-ins per contract, chosen at run time by priority and availability, which is what a microkernel's registry provides.
Event-driven architecture on its own	Showing today's outfit is a request that needs an answer within 2 seconds (NFR-01). Building that from events and handlers would make the main flow hard to follow and test. We use events only for notifications (Table 4).
Client and server with our own backend	It would allow syncing and a server-side model, but it breaks on-device privacy (NFR-02), needs hosting and user accounts, and adds a second codebase. iCloud sync is out of scope this term (FR-21).
Model, View, Controller (MVC) for the screens	SwiftUI has no view controllers, so in practice the logic ends up inside the views, where it is hard to test. MVVM fits SwiftUI's `@Observable` data binding directly.

3 Quality attributes and how we will test them

Table 6 shows how the architecture meets each non-functional requirement in scope, and how and when we will check it. Every test runs in the sprint named, and its result is recorded in the sprint's test log in the GitHub repository (`docs/test-log.md`).

Requirement	How the architecture addresses it	Test: when, how, and the pass mark
NFR-01 Outfit visible within 2 s	The engine's cache returns today's outfit at once. The pipeline runs off the main thread. Recommenders other than the rule-based one get at most 1.5 s, and context sources 3 s each.	Sprint 2. Launch-to-outfit time measured with Instruments (os_signpost markers) on the oldest iPhone the team has, with a 200-item test wardrobe created by `TestWardrobeFactory`. Pass: under 2 s in 9 of 10 cold launches.
NFR-02 Data stays on the device	Every store is local. The only plug-in that sends wardrobe data is `GeminiRecommender`, it sends attributes only, and it is unavailable without consent. Weather plug-ins send only an approximate location.	Sprint 3. Run the app through the Proxyman proxy with consent off and on. Pass: with consent off, only weather requests appear; with it on, the Gemini request contains none of the fields listed as never sent (section 2.3).
NFR-03 Works offline	The rule-based recommender is always the last plug-in and needs no network. The context builder falls back to the saved forecast or to defaults.	Every sprint demo, plus an XCTest that registers only failing context sources. Pass: in airplane mode the wardrobe opens and an outfit appears.
NFR-04	Two tabs (Today, Wardrobe). The Today screen shows an Add item	End of Sprint 2. Five people from outside the team, given a phone with an empty wardrobe and no

Requirement	How the architecture addresses it	Test: when, how, and the pass mark
First outfit within 5 minutes	button, not an error, until a top, a bottom and shoes exist.	instructions. Pass: at least 4 of 5 see an outfit within 5 minutes.
NFR-05 Under 1 s for 200 items	The shortlist of five per slot caps the work at 625 combinations (section 2.2). The scorer is a pure Swift function.	From Sprint 2, an XCTest `measure` test on 200 generated items, run by GitHub Actions on every pull request. Pass: average under 1 s.
NFR-06 No data lost	Photo first, item second, with rollback. Files in Application Support, which iOS does not clear.	End of every sprint: force-quit during a save, and install the new build over the previous one. Pass: every item and photo is still there and no orphan files exist.
NFR-07 Permissions on first use	Each plug-in asks for its own permission the first time it is used and reports itself unavailable if it is denied. The core treats an unavailable plug-in the same as a failed one.	End of Sprints 2 and 3: deny camera, location and calendar in turn. Pass: the app still adds items from the library and still shows an outfit, with the matching note.
NFR-08 Swappable recommender	Recommenders are plug-ins behind one contract. The ViewModel never knows which one answered.	From Sprint 2, an XCTest registers a fake recommender with a higher priority. Pass: its outfit is shown without any change to the app's code.

Table 6. Non-functional requirements, the decisions that meet them, and the test plan.

NFR-09 to NFR-11 are not formally tested this term, but the design supports them. Standard SwiftUI controls work with VoiceOver and Dynamic Type, and every item photo gets an accessibility label made from its category and colour. Strings go in a String Catalog so Arabic can be added later. The deployment target is iOS 17, which SwiftData and `@Observable` need anyway.

4 Architectural and design patterns

Four architectural patterns shape the app: the microkernel as a whole, pipe and filter inside the engine, publish and subscribe for events, and MVVM for the screens. Six design patterns shape individual parts. For each one, the table gives what the pattern is, where we use it, and why it suits Quickdress.

4.1 Architectural patterns

Microkernel (plug-in architecture)

What it is	A minimal core system holds the stable rules and a registry. Optional, replaceable features are plug-ins that implement contracts defined by the core.
Where it is used	`QuickdressCore` with `PluginRegistry`, and the five kinds of plug-in contract and their plug-ins (Tables 2 and 3, Figure 1).
Why it fits	Quickdress's variability is in its interchangeable parts: two weather sources, two recommenders with a fallback, two image sources and four filters, with more already in the backlog. Each becomes a plug-in, so adding one (for example `OpenMeteoSource` if WeatherKit is not available) changes no core code and no screen.

Pipe and filter

What it is	Data passes through a sequence of independent stages, each of which takes input and produces output for the next stage.
Where it is used	The outfit engine: candidates, then SeasonFilter, WeatherFilter, FormalityFilter and TransportComfortFilter, then the recommenders, then validation (section 2.2, Figure 3).
Why it fits	Each rule of the recommendation is a small stage that can be tested alone with a handful of items. A new rule, such as the planned laundry filter (FR-20), is one more stage.

Publish and subscribe (event bus)

What it is	Components publish events to a bus without knowing who receives them. Subscribers register for the events they care about.
Where it is used	`EventBus` and the six events in Table 4.
Why it fits	When an item is deleted, the engine, the wardrobe screen and today's outfit all need to react. With events, `WardrobeManager` does not need to know about any of them.

Model, View, ViewModel (MVVM)

What it is	The View displays state, the ViewModel holds that state and the actions on it, and the Model is the data. The View observes the ViewModel and redraws when the state changes.
Where it is used	Every screen in the host app: TodayView with TodayViewModel, WardrobeView and AddItemView with WardrobeViewModel, FiltersView with FiltersViewModel, and SettingsView with SettingsViewModel. ViewModels are `@Observable` classes.
Why it fits	SwiftUI is built around this kind of binding. ViewModels can be tested without any UI, and they depend only on the core's services.

4.2 Design patterns

Strategy

What it is	A family of interchangeable algorithms behind one interface, so the caller can use any of them without knowing which.
Where it is used	The `Recommender` contract, with `RuleBasedRecommender` and `GeminiRecommender`. The `ContextSource` contract, with `WeatherKitSource` and `OpenMeteoSource`.
Why it fits	NFR-08 asks for this directly. The engine picks the strategy at run time from the registry, by priority and availability.

Plugin and Registry

What it is	Plugin links an implementation to an interface at configuration time rather than in the code that uses it. A registry is a well-known object that other objects use to find those implementations.
Where it is used	`QuickdressApp` registers every plug-in in one place. `PluginRegistry.available(Recommender.self)` returns the available recommenders in priority order.

Why it fits	The choice of plug-ins is made in one file, so the order of the fallback and the set of filters is configuration, not code spread across classes.

Adapter

What it is	A class that wraps an existing interface so it matches the interface a client expects.
Where it is used	`WeatherKitSource`, `OpenMeteoSource`, `EventKitCalendarSource`, `CameraSource`, `PhotoLibrarySource` and `FirebaseGeminiClient` wrap Apple, Google and Open-Meteo interfaces and convert their types into our `Forecast`, `CalendarEvent` and JPEG data.
Why it fits	SDK types never reach the core, which is why the core can stay Foundation-only.

Repository

What it is	An object that behaves like a collection of domain objects and hides how they are stored.
Where it is used	The `ItemStore` contract and `SwiftDataItemStore`, which keeps its own `@Model` records and maps them to the domain types.
Why it fits	All queries and the SwiftData `ModelContext` live in one plug-in. Tests use an in-memory store. Moving to Core Data (risk R8) or CloudKit (FR-21) means writing one new plug-in.

Observer

What it is	An object notifies its observers automatically when its state changes.
Where it is used	SwiftUI views observe their `@Observable` ViewModels. The event bus applies the same idea between the core and the ViewModels.
Why it fits	It keeps the screens in step with the data without the core ever calling a view.

Dependency injection

What it is	Objects receive the things they depend on from outside instead of creating them.
Where it is used	`QuickdressApp` creates the registry, the event bus, the plug-ins and the core services, and passes the services to the ViewModels through their initialisers.
Why it fits	It lets tests and previews use fakes for every plug-in, which is how the tests in Table 6 run without a network or real permissions.

5 Detailed design: class diagram

Quickdress: UML class diagram

Microkernel design. Core services (green), plug-in contracts (orange), plug-ins (teal), host app (purple), value types (blue), enumerations and type aliases (grey). Each frame is one Swift package.

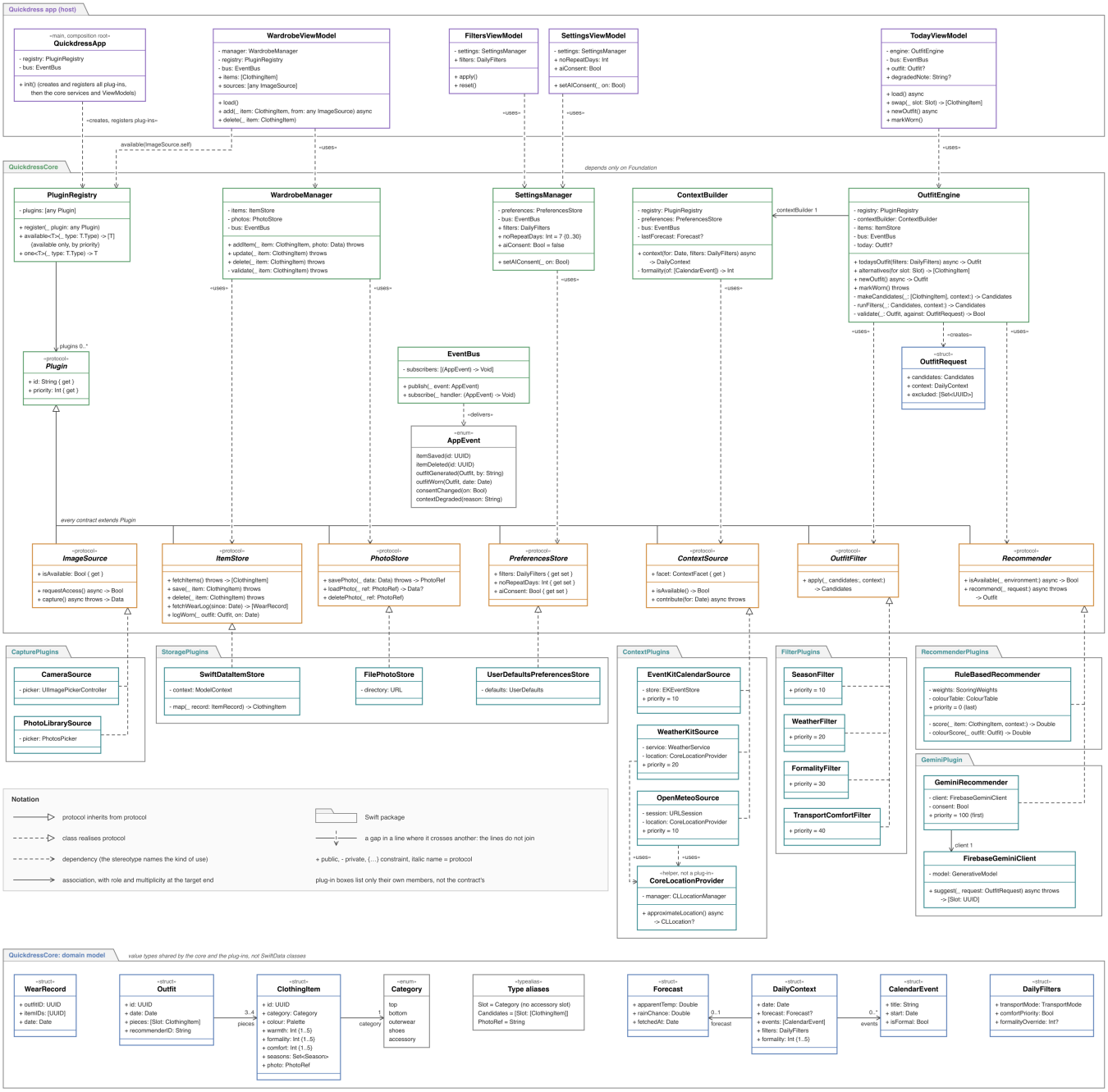


Figure 4. UML class diagram (Swift types). The components in Figure 1 are made up of these classes.

Table 7 maps each component in Figure 1 to its classes in Figure 4, and names the Swift package each belongs to.

Component	Package	Classes in Figure 4
PluginRegistry	QuickdressCore	Plugin, PluginRegistry
EventBus	QuickdressCore	EventBus, AppEvent
WardrobeManager	QuickdressCore	WardrobeManager
ContextBuilder	QuickdressCore	ContextBuilder
OutfitEngine	QuickdressCore	OutfitEngine, OutfitRequest
SettingsManager	QuickdressCore	SettingsManager
Plug-in contracts	QuickdressCore	ImageSource, ItemStore, PhotoStore, PreferencesStore, ContextSource, OutfitFilter, Recommender
Domain model	QuickdressCore	ClothingItem, Outfit, WearRecord, DailyContext, DailyFilters, Forecast, CalendarEvent, Category, Slot
Capture plug-ins	CapturePlugins	CameraSource, PhotoLibrarySource (and ItemAnalyzer, a stretch helper used by the Add item screen, not drawn)
Storage plug-ins	StoragePlugins	SwiftDataItemStore, FilePhotoStore, UserDefaultsPreferencesStore
Context plug-ins	ContextPlugins	WeatherKitSource, OpenMeteoSource, EventKitCalendarSource, CoreLocationProvider
Filter plug-ins	FilterPlugins	SeasonFilter, WeatherFilter, FormalityFilter, TransportComfortFilter
Recommender plug-ins	RecommenderPlugins, GeminiPlugin	RuleBasedRecommender, GeminiRecommender, FirebaseGeminiClient
Screens (host app)	Quickdress app	TodayViewModel, WardrobeViewModel, FiltersViewModel, SettingsViewModel, QuickdressApp

Table 7. Mapping from components to packages and classes.

`GeminiRecommender` has its own package so that the Firebase SDK is linked only into that package. Every contract is drawn with the plug-ins that realise it. Dependencies that a class holds as a typed attribute, such as `OutfitEngine.registry`, are listed in the class box; only the main ones are drawn as arrows to keep the diagram readable.

6 Technical feasibility, build order and risks

6.1 Feasibility

Every in-scope feature uses well-documented first-party Apple frameworks: SwiftUI, SwiftData, PhotosUI, AVFoundation, WeatherKit, EventKit and Core Location, or the free Open-Meteo API. The only third-party SDK is Firebase, used only by the stretch Gemini plug-in. The hardest algorithm, the recommender, is rule-based scoring in plain Swift with the rules written out in section 2.2.

The microkernel adds two components that a layered design does not have: the registry and the event bus. Both are small. The registry is a dictionary from contract to a sorted list of plug-ins, and the bus is a list of handlers per event. We keep them under 150 lines together, and they are built and tested in the first week of Sprint 1, before any plug-in. Because plug-ins are compiled in, not loaded at run time, there is no dynamic loading to get right.

6.2 Build order

Each sprint ends at the next course milestone and delivers a working app that can be demonstrated. Table 8 lists what each sprint builds and the exit criteria that must pass before the sprint counts as done. Each person owns whole plug-ins, which is possible because plug-ins do not depend on each other.

Sprint	Components built	Exit criteria
Sprint 1	Core: PluginRegistry, EventBus, WardrobeManager, domain model. Plug-ins: CameraSource, PhotoLibrarySource, SwiftDataItemStore, FilePhotoStore. Host app: Wardrobe screens.	FR-01 to FR-04 shown in the demo. Unit tests for WardrobeManager (validation and photo rollback) pass with in-memory stores. The NFR-06 test passes.
Sprint 2	Core: ContextBuilder, OutfitEngine, SettingsManager. Plug-ins: OpenMeteoSource (and WeatherKitSource if R1 allows), SeasonFilter, WeatherFilter, RuleBasedRecommender, UserDefaultsPreferencesStore. Host app: Today screen.	FR-05 to FR-08 shown in the demo. The NFR-01, NFR-03, NFR-04, NFR-05, NFR-07 and NFR-08 tests in Table 6 pass.
Sprint 3	Plug-ins: EventKitCalendarSource, FormalityFilter, TransportComfortFilter. Core: wear log and no-repeat period. Host app: Filters screen.	FR-09 to FR-11 shown in the demo. The NFR-02, NFR-03, NFR-06 and NFR-07 tests pass.
Sprint 3 stretch	In this order: saved outfits (FR-12), ItemAnalyzer (FR-13), GeminiRecommender and the Settings screen (FR-22).	Started only if every Sprint 2 exit criterion has passed when Sprint 3 begins. A stretch item that is not merged one week before the Sprint 3 demo is dropped. ItemAnalyzer ships only if it suggests the right category for at least 24 of 30 of the team's own items.

Table 8. Build order and exit criteria by sprint.

Every change goes through a pull request on GitHub. The `main` branch is protected: a pull request needs one approving review from another team member and a passing GitHub Actions run (build plus all tests) before it can be merged.

6.3 Risks

ID	Risk	Chance	Impact	Mitigation	Decided by
R1	WeatherKit needs a paid Apple Developer Program membership ($99 a year). Without one, test installs also expire every 7 days.	High	High: blocks FR-08	`OpenMeteoSource` (free for non-commercial use, no key) is built in Sprint 2 regardless. `WeatherKitSource` is added only if a membership exists, as a higher-priority plug-in.	Start of Sprint 2

ID	Risk	Chance	Impact	Mitigation	Decided by
R2	Xcode runs only on macOS. A team member without a Mac cannot build or run the app.	Medium	High	Pair those members with Mac owners, or give them work that needs no Xcode: the colour table, the keyword list, the 200-item test data and the test plans in Table 6.	Start of Sprint 1
R3	The team is learning Swift and SwiftUI during the project.	High	Medium	Sprint 1 is plain create, read, update and delete work. Each plug-in is small and has one owner. Every pull request is reviewed.	Ongoing
R4	The plug-in structure is new to the team and could slow Sprint 1.	Medium	Medium	Registry and bus under 150 lines, built first with tests. No dynamic loading. After the Sprint 1 review, a change to a contract needs the agreement of the whole team in the pull request.	End of Sprint 1 week 1
R5	Gemini replies may be wrong, slow or not valid JSON.	Medium	Low	Structured output, the engine's validation, a 1.5 s limit and the rule-based fallback.	Sprint 3 (stretch)
R6	Gemini cost, quota or model names change.	Low	Low	Flash-Lite is Google's lowest-cost model in the family. The model name is in Remote Config. At most one call per generated outfit, and none for swaps.	Sprint 3 (stretch)
R7	Detecting formality from calendar titles by keyword misses vague titles such as "Meeting".	High	Medium	The user can tag an event as formal, and the formality override always wins. The keyword list (section 2.2) is in one file.	Sprint 3
R8	SwiftData is fairly new and has known bugs, especially with schema changes.	Medium	Medium	SwiftData is used only inside `SwiftDataItemStore`. If it fails the NFR-06 test twice, we replace that one plug-in with a Core Data version.	End of Sprint 1
R9	Automatic category and colour suggestion (FR-13) may be inaccurate.	Medium	Low	Stretch goal with a pass mark of 24 correct out of 30 (Table 8). The user always confirms the suggestion.	Sprint 3 (stretch)
R10	The iOS Simulator has no camera.	Certain	Low	`CameraSource` reports itself unavailable on the Simulator, so the photo library is used there. The camera is tested on a real phone.	Sprint 1
R11	Event handlers could run in an unexpected order or trigger each other in a loop.	Low	Medium	Handlers run in subscription order on the main thread, may not publish the same event type, and each event has a unit test.	Sprint 1

Table 9. Technical risks.

Core ML is no longer part of this term's plan. We have no training data, so a model could not be tested. The design keeps the option open: an on-device recommender would be one more plug-in (Table 3).

7 Conclusion

Quickdress is built as a microkernel. A small, Foundation-only core holds the wardrobe rules, the context builder, the outfit engine, a plug-in registry and an event bus. Everything that varies is a plug-in: image sources, stores, weather and calendar sources, outfit filters and recommenders. Inside the engine, a pipe-and-filter pipeline narrows the wardrobe, and recommenders are tried by priority with a rule-based plug-in that always answers, which is how the design meets the offline and privacy requirements while still allowing the opt-in Gemini step. The screens use MVVM and talk only to the core. Section 2.2 writes out the recommendation rules, Table 6 gives a test with a pass mark for each of NFR-01 to NFR-08, and Table 8 sets exit criteria for each sprint and a rule for when stretch goals are built.